

Name : Fatouma Hassan Elmi

Week 3 SQL Analysis Report

I- Chinook Database (Music Store)

1. Introduction

The Chinook Database represents a digital music store containing information about customers, artists, albums, tracks, invoices, and sales transactions.

The objective of this analysis is to practice SQL querying skills by extracting, transforming, and analyzing structured data using PostgreSQL.

The database was imported into PostgreSQL using pgAdmin, where all SQL queries were executed.

The analysis covers:

- Data retrieval using SELECT statements
- Filtering and sorting data
- Aggregation and grouping
- Joining multiple tables
- Advanced SQL concepts such as subqueries and window functions

2. Core SQL Queries and Insights

2.1 Customer Information Retrieval (SELECT)

Query Purpose

The SELECT statement was used to retrieve customer information from the customer table.

Query:

```
SELECT  
first_name,  
last_name,  
email  
FROM customer;
```

Explanation:

This query extracts customer names and email addresses to understand the available customer records.

Insight:

The query provides an overview of the customer database and available customer contact information.

2.2 Filtering Customers by Country (WHERE)

Query Purpose

Identify customers located in Brazil.

Query:

```
SELECT
first_name,
last_name,
country
FROM customer
WHERE country = 'Brazil';
```

Explanation:

The WHERE clause filters records based on a specific condition.

Insight:

This query demonstrates how customer data can be segmented geographically for targeted analysis.

2.3 Sorting Customers (ORDER BY)

Query Purpose

Arrange customers alphabetically by last name.

Query:

```
SELECT
first_name,
last_name
FROM customer
ORDER BY last_name ASC;
```

Explanation:

The ORDER BY clause sorts the returned data in ascending order.

Insight:

Sorting improves readability and helps organize customer information efficiently.

3. Aggregate Analysis

3.1 Total Number of Customers (COUNT)

Query:

```
SELECT COUNT(*) AS total_customers
FROM customer;
```

Explanation:

The COUNT function calculates the total number of customer records.

Insight:

The Chinook database contains 59 customers, representing the size of the store's customer base.

total_customers
59

3.2 Total Revenue (SUM)

Query:

```
SELECT SUM(total) AS total_revenue
FROM invoice;
```

Explanation:

The SUM function calculates the total revenue generated from invoices.

Insight:

This provides an overview of the overall sales performance of the music store.

total_revenue
2328.6

3.3 Average Invoice Value (AVG)

Query:

```
SELECT AVG(total) AS average_invoice
FROM invoice;
```

Explanation:

The AVG function calculates the average transaction value.

Insight:

The average invoice value helps understand typical customer spending behavior.

average_invoice
5.6519417475728155

4. Grouping and Business Analysis

4.1 Customers by Country (GROUP BY)

Query:

```
SELECT
country,
COUNT(*) AS number_of_customers
FROM customer
GROUP BY country;
```

Explanation:

The GROUP BY clause groups customers according to their country.

Insight:

This analysis shows the geographic distribution of customers and identifies important markets.

4.2 Countries with More Than 5 Customers (HAVING)

Query:

```
SELECT
country,
COUNT(*) AS number_of_customers
FROM customer
```

```
GROUP BY country
HAVING COUNT(*) > 5;
```

Explanation:

The HAVING clause filters grouped results based on aggregate values.

Insight:

This identifies countries with a larger customer presence, which may represent stronger markets.

5. Advanced SQL Analysis**5.1 Customer Purchase History (INNER JOIN)****Query Purpose:**

Combine customer information with invoice records.

Query:

```
SELECT
    c.first_name,
    c.last_name,
    i.invoice_id,
    i.invoice_date,
    i.total
FROM customer c
INNER JOIN invoice i
ON c.customer_id = i.customer_id;
```

Explanation:

The INNER JOIN connects the customer table and invoice table using customer_id.

Insight:

This allows analysis of customer purchasing activity and spending behavior.

5.2 Above Average Transactions (Subquery)**Query Purpose:**

Find invoices higher than the average invoice value.

Query:

```
SELECT
    invoice_id,
    total
FROM invoice
WHERE total >
(
    SELECT AVG(total)
    FROM invoice
);
```

Explanation:

The subquery first calculates the average invoice amount, then retrieves invoices exceeding that value.

Insight:

These transactions represent higher-value purchases and can help identify valuable sales.

5.3 Customer Ranking (Window Function)**Query Purpose:**

Rank customers based on spending.

Query:

```
SELECT
    c.first_name,
    c.last_name,
    SUM(i.total) AS total_spent,
    RANK() OVER (ORDER BY SUM(i.total) DESC) AS customer_rank
FROM customer c
JOIN invoice i
ON c.customer_id = i.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name;
```

Explanation:

The RANK() window function assigns rankings according to total spending.

Insight:

This helps identify top customers and supports customer segmentation strategies.

6. Recommendations

- Focus marketing campaigns on countries with the highest customer counts.
- Reward high-spending customers through loyalty programs.
- Monitor revenue trends regularly to identify sales opportunities.
- Analyze customer purchase history to recommend similar music products.

7. Conclusion

The Chinook database analysis demonstrated how SQL can be used to extract meaningful business insights from relational data.

The queries helped analyze:

- Customer information
- Revenue performance
- Customer distribution
- Purchasing behavior
- High-value transactions

These SQL techniques are essential skills for data analysts working with real-world databases.

II- Sales Dataset (AdventureWorks Lite / Equivalent)

1. Introduction

This report presents the SQL analysis performed on the Sales Data dataset using PostgreSQL. The objective was to practice SQL querying techniques, analyze sales performance, identify business trends, and demonstrate query optimization techniques.

2. Core SQL Queries

2.1 Retrieve Sales Information

Query

```
SELECT
    order_number,
    product_line,
    sales,
    status
FROM sales_data;
```

Explanation

This query retrieves the order number, product line, sales amount, and order status for every transaction in the dataset.

Insight

The query provides an overview of the sales records and helps understand the structure of the dataset before performing further analysis.

2.2 Retrieve Shipped Orders

Query

```
SELECT
    order_number,
    product_line,
    sales
FROM sales_data
WHERE status = 'Shipped';
```

Explanation

The WHERE clause filters the dataset to display only orders that have been successfully shipped.

Insight

This allows the business to focus on completed transactions when calculating revenue and evaluating sales performance.

2.3 Sort Orders by Sales

Query

```
SELECT
  order_number,
  product_line,
  sales
FROM sales_data
ORDER BY sales DESC;
```

Explanation

The ORDER BY clause sorts the orders from the highest sales value to the lowest.

Insight

This helps identify high-value transactions and supports revenue analysis.

2.4 Count Total Orders

Query

```
SELECT COUNT(*) AS total_orders
FROM sales_data;
```

Explanation

The COUNT() function calculates the total number of sales records.

Insight

Knowing the total number of orders provides a basic measure of business activity.

3. Aggregate Functions

3.1 Calculate Total Revenue

Query

```
SELECT
  ROUND(SUM(sales),2) AS total_revenue
FROM sales_data;
```

Explanation

The SUM() function calculates the total revenue generated from all sales.

Insight

Total revenue is one of the most important business performance indicators.

3.2 Calculate Average Sale

Query

```
SELECT
  ROUND(AVG(sales),2) AS average_sale
```

```
FROM sales_data;
```

Explanation

The AVG() function computes the average sales amount per transaction.

Insight

The average sale value helps measure customer spending patterns.

3.3 Revenue by Product Line

Query

```
SELECT
    product_line,
    ROUND(SUM(sales),2) AS total_revenue
FROM sales_data
GROUP BY product_line
ORDER BY total_revenue DESC;
```

Explanation

The GROUP BY clause groups records according to product line and calculates total revenue for each group.

Insight

This analysis identifies which product categories contribute the most revenue.

3.4 Product Lines with High Revenue

Query

```
SELECT
    product_line,
    ROUND(SUM(sales),2) AS total_revenue
FROM sales_data
GROUP BY product_line
HAVING SUM(sales) > 100000
ORDER BY total_revenue DESC;
```

Explanation

The HAVING clause filters grouped results to include only product lines generating more than \$100,000 in revenue.

Insight

This highlights the strongest-performing product categories.

4. Advanced SQL

4.1 Orders Above Average Sales

Query


```
SELECT
    order_number,
    product_line,
    sales
FROM sales_data
WHERE sales >
(
    SELECT AVG(sales)
    FROM sales_data
);
```

Explanation

The subquery calculates the average sales value, while the main query returns only orders above that average.

Insight

Orders exceeding the average sale value represent high-value transactions that may deserve further business attention.

5. Window Functions

5.1 Rank Orders by Sales

Query

```
SELECT
    order_number,
    product_line,
    sales,
    RANK() OVER (
        ORDER BY sales DESC
    ) AS sales_rank
FROM sales_data;
```

Explanation

The RANK() window function assigns a ranking to each order according to its sales amount.

Insight

Ranking helps identify the highest-value orders without removing any records.

5.2 Rank Orders Within Each Product Line

Query

```
SELECT
    product_line,
    order_number,
    sales,
    RANK() OVER (
        PARTITION BY product_line
```

```
ORDER BY sales DESC
) AS product_rank
FROM sales_data;
```

Explanation

This query ranks orders separately within each product line.

Insight

It allows comparison of the best-performing orders within each product category.

6. Business Problem Solving

6.1 Top-Performing Product Lines

Query

```
SELECT
    product_line,
    ROUND(SUM(sales),2) AS total_revenue
FROM sales_data
GROUP BY product_line
ORDER BY total_revenue DESC;
```

Explanation

This query calculates the total revenue generated by each product line.

Insight

The product line with the highest revenue is the company's strongest-performing category and may deserve increased marketing or inventory investment.

6.2 Revenue Trends Over Time

Query

```
SELECT
    year_id,
    month_id,
    ROUND(SUM(sales),2) AS monthly_revenue
FROM sales_data
GROUP BY year_id, month_id
ORDER BY year_id, month_id;
```

Explanation

This query summarizes monthly revenue by year.

Insight

Monthly revenue trends help identify seasonality, growth patterns, and periods of high or low business performance.

6.3 Quarterly Revenue

Query

```
SELECT
    year_id,
    qtr_id,
    ROUND(SUM(sales),2) AS quarterly_revenue
FROM sales_data
GROUP BY year_id, qtr_id
ORDER BY year_id, qtr_id;
```

Explanation

This query calculates total revenue for each quarter.

Insight

Quarterly analysis supports strategic planning and performance evaluation throughout the year.

6.4 Order Analysis

Query

```
SELECT
    order_number,
    COUNT(*) AS products_ordered,
    ROUND(SUM(sales),2) AS total_order_value
FROM sales_data
GROUP BY order_number
ORDER BY total_order_value DESC;
```

Explanation

The query summarizes each order by counting the number of products purchased and calculating its total value.

Insight

High-value orders may indicate bulk purchases or premium product sales and can help identify important customer purchasing patterns.

6.5 Revenue by Order Status

Query

```
SELECT
    status,
    COUNT(*) AS total_orders,
    ROUND(SUM(sales),2) AS revenue
FROM sales_data
GROUP BY status
ORDER BY revenue DESC;
```

Explanation

This query compares the number of orders and revenue generated by each order status.

Insight

Comparing shipped, cancelled, and pending orders helps evaluate operational efficiency and identify potential revenue losses.

7. Query Optimization

Create an Index

Query

```
CREATE INDEX idx_product_line  
ON sales_data(product_line);
```

Explanation

An index improves the speed of queries that search or filter by `product_line`.

Insight

Indexing reduces the time required to retrieve data, especially for large datasets.

Analyze Query Performance

Query

```
EXPLAIN ANALYZE  
  
SELECT  
    order_number,  
    product_line,  
    sales  
FROM sales_data  
WHERE product_line = 'Classic Cars';
```

Explanation

`EXPLAIN ANALYZE` displays PostgreSQL's execution plan and measures query performance.

Insight

The execution plan helps determine whether PostgreSQL uses an index or performs a full table scan, allowing optimization of slow queries.

8. Conclusion

The SQL analysis of the Sales Data dataset demonstrated the use of fundamental and advanced SQL concepts, including filtering, aggregation, subqueries, window functions, business analysis, and query optimization. The analysis identified revenue-generating product lines, evaluated sales trends over time, ranked high-value transactions, and explored order patterns. Finally, indexing and execution plan analysis illustrated how SQL queries can be optimized to improve performance on large datasets. This exercise strengthened practical SQL skills while providing meaningful business insights from transactional sales data.

